浙江大学

本科实验报告

课程名称：基于 K-means 聚类的图像颜色量化

姓　　名：

学　　院：信息与电子工程学院

　　系：信息与电子工程学院

专　　业：信息工程

学　　号：

指导教师：程磊

2025 年 11 月 11 日

专业：信息工程
姓名：
学号：
日期：2025.11.11

浙江大学实验报告

课程名称：电子电路设计实验 I　　指导老师：程磊　　成绩：____
实验名称：基于 K-means 聚类的图像颜色量化

一、问题背景

彩色图像通常使用 RGB 模型存储颜色，每个像素可以看作是三维空间中的一个点。颜色量化的目标是将图像中使用的颜色数量减少到一个较小的预设值 K，同时尽量保持原始图像的视觉外观。

本次课程 project 中，我们假设一张图像可以用$H * W * 3$的数组表示，亦可将其视为 $H * W$ 矩阵，其中每一个元素是一个 RGB 三通道的向量，分别是图片的高度和宽度。从零开始实现 K-means 聚类算法，将其应用于图像像素的 RGB 向量，找到 K 个“代表性颜色”（聚类中心）。然后，用这些代表性颜色替换原始像素颜色，实现一幅图像的量化。

二、K-means 理论分析

1．K-means 介绍：聚类问题中我们仅仅知道数据点本身，而不知道每个数据点具体的标签。聚类分析的任务就是将一些无标签的数据点按照某种相似度来进行归类，进而从数据点本身来学习其内蕴的类别特征，因此是一种经典的无监督学习任务

2．数学分析：

给定 p 维空间中$a_1, a_2, ..., a_n \in R^p$个数据点，假定两个数据点之间的相似度可以通过其欧几里得距离来衡量．我们的目标是将相似的点归为一类，同时将不相似的点区分开。

假设待聚类的个数已知，记为 K，且同一数据点只属于一个类，因此聚类问题转化为寻找 K 个不相交的非空集合：$S_1, S_2, ..., S_K$使得：

$$\{a_1, a_2, ..., a_n\} = S_1 \cup S_2 \cup \cdots \cup S_K$$

采用欧氏距离描述同类点之间的远近，定义组内距离平方和 WCSS：

$$W(S_1, S_2, ..., S_k) = \sum_{i=1}^{K} \sum_{a \in S_i} \| a - c_i \|^2$$

其中 ci 为第 i 类数据点的中心点，建立 model，找到一个聚类方式，使得 WCSS 最小，且这 K 类将所有数据包含：

$$\min_{S_1, S_2, ..., S_K} \sum_{i=1}^{K} \sum_{a \in S_i} \| a - c_i \|^2$$

$$\text{s.t. } S_1 \cup S_2 \cup \cdots \cup S_K = \{a_1, a_2, ..., a_n\},\ S_i \cap S_j = \emptyset, \forall i \neq j.$$

3．问题一：数学建模问题的转化

若定义矩阵$H \in \mathbb{R}^{K \times p}$，且第 i 行的向量是$h_i^T$，用向量$\phi_i \in \mathbb{R}^K$表示 ai 所处的类别，定义：

$$(\phi_i)_j = \begin{cases} 0, & a_i \notin S_j \\ 1, & a_i \in S_j \end{cases}$$

则聚类问题等价描述为：

$$\min_{\Phi, H} \| A - \Phi H \|_F^2$$

$$s.t. \Phi \in \mathbb{R}^{n \times K}，且每一行只有一个元素是 1，其余是 0$$

$$H \in \mathbb{R}^{K \times p}, A的第i行是a_i^T.$$

Proof：该优化问题与原始的 $\min \sum_{i=1}^{K} \sum_{a \in S_i} \|a - c_i\|^2$问题表述等价

（1）理解$||A - \Phi H||_F^2$：属于矩阵的 F 范数，是每个元素的平方和，A为 n * p 的矩阵，Φ为 n * K 矩阵，由于每一行只有一个元素是 1，其余是 0，属于 one-hot 编码，H为 K * p 的矩阵

（2）分析$A - \Phi H$的第 i 行：因为ϕ_i^T是 one-hot（假设第 j 个元素为 1），$\phi_i^T H$的结果就是H的第 j 行，即h_j^T，则$(A - \Phi H)_i = \mathrm{a}_i^T - h_j^T$，j 为类别 K 之一

（3）$||A - \Phi H||_F^2 = \sum_{i=1}^{n} ||(A - \Phi H)的第i行||^2 = \sum_{j=1}^{\mathrm{K}} |\sum_{i=1}^{n}| |\mathrm{a}_i^T - h_j^T||^2 = \sum_{j=1}^{K} \sum_{a \in S_j} \| a - h_j \|^2$，也就等价于有 K 类的$h_j^T$，将$\mathrm{a}_i^T$分为 K 类后，优化所有距离和的求和，使其最小，因此等价于$\min \sum_{i=1}^{K} \sum_{a \in S_i} \|a - c_i\|^2$

$$A-\Phi H=\begin{bmatrix}a_1^T\\a_2^T\\\vdots\\a_n^T\end{bmatrix}-\begin{bmatrix}\phi_1^T\\\phi_2^T\\\vdots\\\phi_n^T\end{bmatrix}H=\begin{bmatrix}a_1^T-\phi_1^TH\\a_2^T-\phi_2^TH\\\vdots\\a_n^T-\phi_n^TH\end{bmatrix}=\begin{bmatrix}a_1^T-h_{j1}^T\\a_2^T-h_{j2}^T\\\vdots\\a_n^T-h_{jK}^T\end{bmatrix}=\begin{bmatrix}a_1^T-c_{j1}^T\\a_2^T-c_{j2}^T\\\vdots\\a_n^T-c_{jK}^T\end{bmatrix}$$

（4）证明参考点集H的取法实际上就是每一类的中心点：详细见问题二中的证明，即证明h_j^T实际为属于这一类的a_i^T的平均值

三、BCD 算法应用于 Kmeans

1．BCD 算法简介：

目标函数的变量联合求解目标函数的极小值通常很困难，但这些自变量具有某种“可分离”的形式：当固定其中若干变量时，函数的结构会得到极大的简化.这种特殊的形式使得人们可以将原问题拆分成数个只有少数自变量的子问题

$$\min_{x\in X}F(x_1,x_2,...,x_n)=f(x_1,x_2,...,x_n)+\sum_{i=1}^{n}r_i(x_i)$$

可以按照下面的方式执行BCD：按照 x1，x2，...，xn 的次序依次固定其余 n-1 个变量块极小化 ，完成一块变量的极小化后，它的值便立即被更新到变量空间中，更新下一块变量时将使用每个变量最新的值

2．用 BCD 求解 Kmeans 问题：

（1）固定 H，设Φ的每一行为ϕ_i^T，则可以得到：

$$A-\Phi H=\begin{bmatrix}a_1^T\\a_2^T\\\vdots\\a_n^T\end{bmatrix}-\begin{bmatrix}\phi_1^T\\\phi_2^T\\\vdots\\\phi_n^T\end{bmatrix}H=\begin{bmatrix}a_1^T-\phi_1^TH\\a_2^T-\phi_2^TH\\\vdots\\a_n^T-\phi_n^TH\end{bmatrix}$$

注意到ϕ_i^T中只有一个为 1，one-hot，设第 j 个分量为 1，容易证明，j 应该取 H 中距离a_i^T最近的一行

$$\Phi_{ij}=\begin{cases}0，\ 其他\\1，\ j=\underset{l}{\operatorname{argmin}}||a_i-h_l||\end{cases}$$

（2）固定Φ，考虑 H 的每一行h_j^T，则

$$\|A-\Phi H\|_F^2=\sum_{j=1}^{K}\sum_{a\in S_j}\|a-h_j\|^2$$

问题二：设$\overline{a_j}$是目前第 j 类的所有点的均值，证明：此时h_j直接取为$\overline{a_j}$即可达到最小值：

①目标函数：$J(h_j)=\sum_{a\in S_j}||a-h_j||^2$，求证此时$h_j=\overline{a_j}=\frac{1}{num_S_j}\sum_{a\in S_j}\mathrm{a_i}$

②分析：此函数是一个无约束的优化问题，因此直接对h_j求偏导即可

③解：$\nabla_{h_j}J(h_j)=\nabla_{h_j}\sum_{a\in S_j}(a-h_j)^T(a-h_j)=\nabla_{h_j}\sum_{a\in S_j}(a^Ta-2a^Th_j+h_j^Th_j)=0$

解得：$-2\sum_{a\in S_j}a+2\sum_{a\in S_j}h_j=0$

假设S_j中有n_j个点，则有$h_j=\frac{1}{n_j}\sum_{a\in S_j}a=\overline{a_j}$

四、实验流程

1．加载图像。可以用Matlab 的 imread, rgb2gray, imshow 等函数进行读取和画图

2．数据预处理：这一步可以用Matlab 的 im2double 函数把图像从像素值变成的 double 类型数字便于后续处理

3．编写 K-means 算法。这一步需要给定算法输入的待聚类数据矩阵，用戶预先定义的聚类数量 K，以及能够容忍的最大迭代次数 max_iter

（1）初始化：从 N 个像素中随机选取 K 个，将这 K 个索引作为 center 寻找簇像素向量；初始化 loss

```
% 初始化中心
rand_num = randperm(N, K);
center = P(rand_num, :);

loss_history = zeros(max_iter, 1);
```

（2）迭代过程：分为两个过程，分别固定 H 和Φ

①固定 H：先计算所有 a 到这几个 center 的距离，然后选取最小的距离对应的 center 作为这个向量 a 的簇 center，并在这个一步记录 loss

```
% 迭代
for i = 1:max_iter
    % 固定 H，求解 Φ
    % 计算所有点到所有聚类中心的距离
    distances = pdist2(P, center, 'squaredeuclidean');
    % 最近的聚类中心
    [min_distances, idx] = min(distances, [], 2);% idx存储每个像素点所属类
    % Loss
    loss_history(i) = sum(min_distances);
```

②固定Φ：重新更新 H，重新计算属于某个簇中所有 a 矢量的平均值，作为这个簇的 center，如果这个簇中没有 a 矢量，则保持不变

```
%固定 Φ，求解 H
%重新算每个类所有点的均值并作为新的参考点
for k = 1:K
    belong_k = P(idx == k, :);% 找到所有属于第 k 簇的像素点

    % 计算均值作为新的聚类中心
    if ~isempty(belong_k) % 确保第k类中有点
        center(k, :) = mean(belong_k, 1);
    end
end
```

（3）终止程序：将终止条件设置为 loss 不再改变

```
% 是否收敛
if i > 1 && loss_history(i) == loss_history(i-1)
    fprintf('算法在第 %d 次迭代收敛。\n', i);
    loss_history(i+1:end) = loss_history(i);
    break;
end
```

4．可视化结果：使用 im2uint8 函数将聚类后的数据矩阵从[0,1]乘以 255 后四舍五入转换到像素值数据范围，再使用 imshow 函数可视化。

（1）代码：

①创建量化后的数据矩阵

```
% 创建新的像素矩阵 (N x 3)
new_P = center(idx, :);

% Reshape 回原始图像尺寸 (m x n x 3)
new_image_double = reshape(new_P, m, n, p);
```

②将 double 类型 (0.0-1.0) 的图像转换回 uint8 (0-255) 以便显示和保存

```
% Write your own code here.

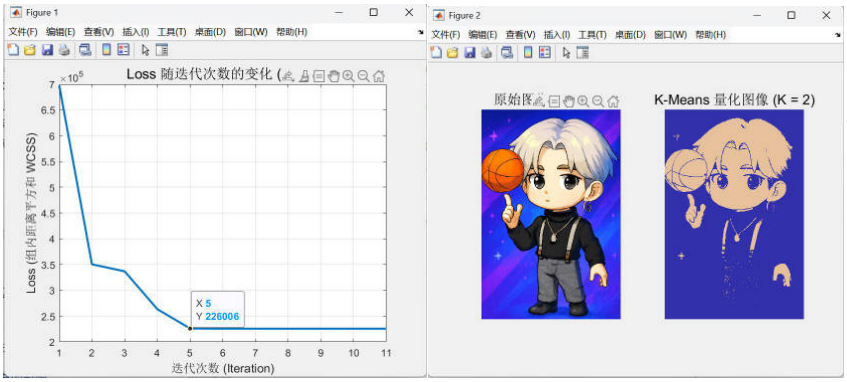
new_image_uint8 = im2uint8(new_image_double);

fprintf('图像重建完成。\n');
```

（2）问题三：

①分析用戶指定的聚类数目K 对 loss 的影响，对图像质量和细节的影响：

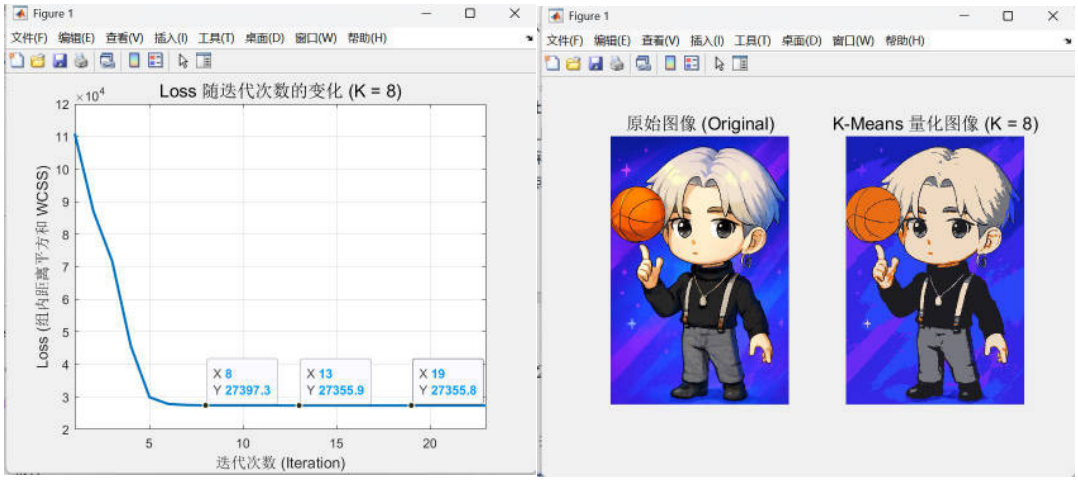
A.K=2 时：loss=226006



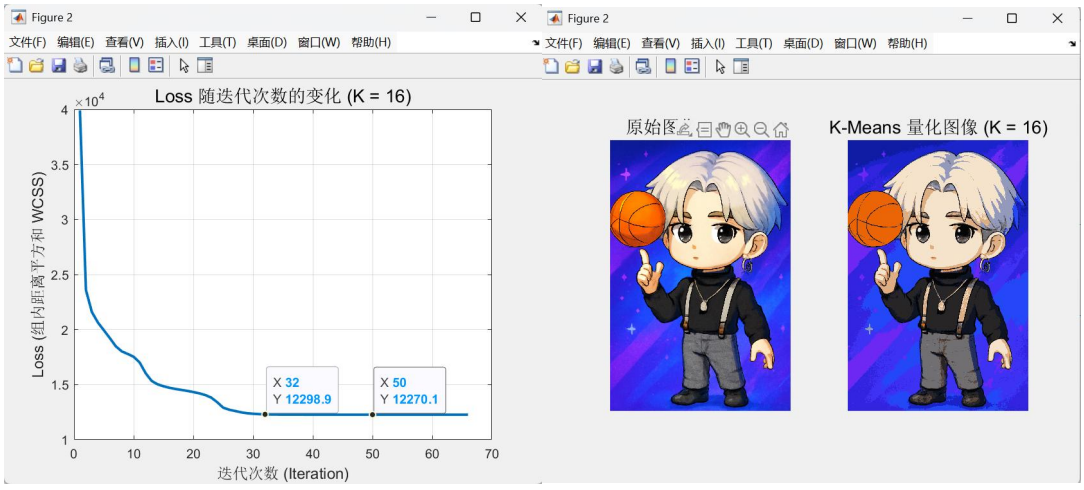
B.K=4 时：loss=64208.7



C.K=8 时：loss=27397.3



D.K=16 时：loss=12298.9



E.观察图像质量：当 K=2 时，图像应该只有两种颜色，细节损失非常严重随着 K 增大 (如 K=8 或 K=16)，图像会使用更多颜色，视觉效果会更接近原

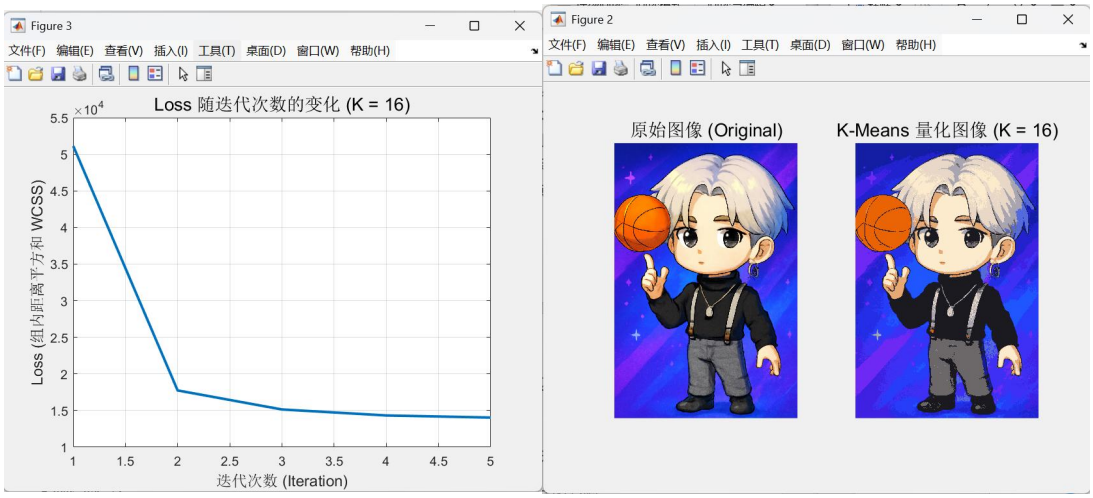
始图像，细节（如篮球上的线条、衣服的阴影）会更丰富。

F.观察 Loss：K 越大，最终的 loss_history(end) 值（即最终 WCSS 值）会越低。这是因为有更多的聚类中心来拟合数据点，每个点到其最近中心的平均距离会更短。

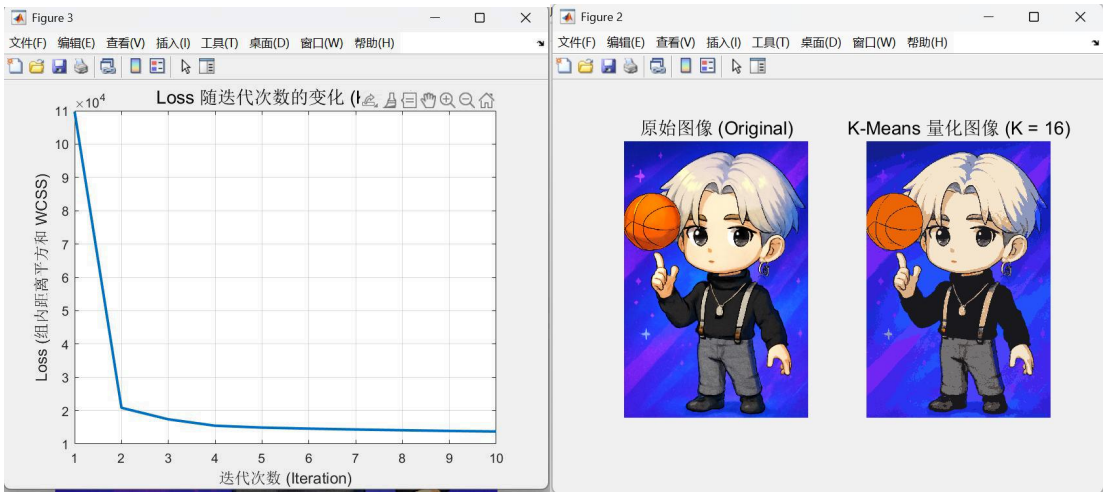
②固定聚类数目K，绘制 loss 对随迭代次数的变化曲线，讨论算法是否单调下降，讨论不同的 max_iter 对收敛行为的影响（包括运行结束的 loss 值，图像视觉效果等。不妨设 K=16

查看代码生成的“Loss 随迭代次数的变化”图。你会发现 Loss 曲线是**单调不减**的（在 K-Means 中是单调下降或保持不变）。这是因为算法的每一步（分配簇、更新中心）都是在最小化总的 WCSS 。

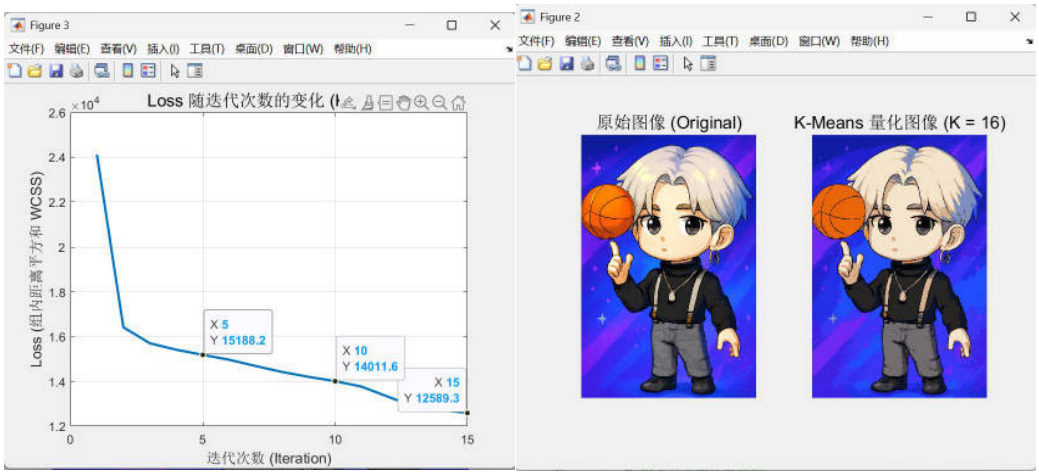
A.max_iter=5



B.max_iter=10



C.max_iter=15



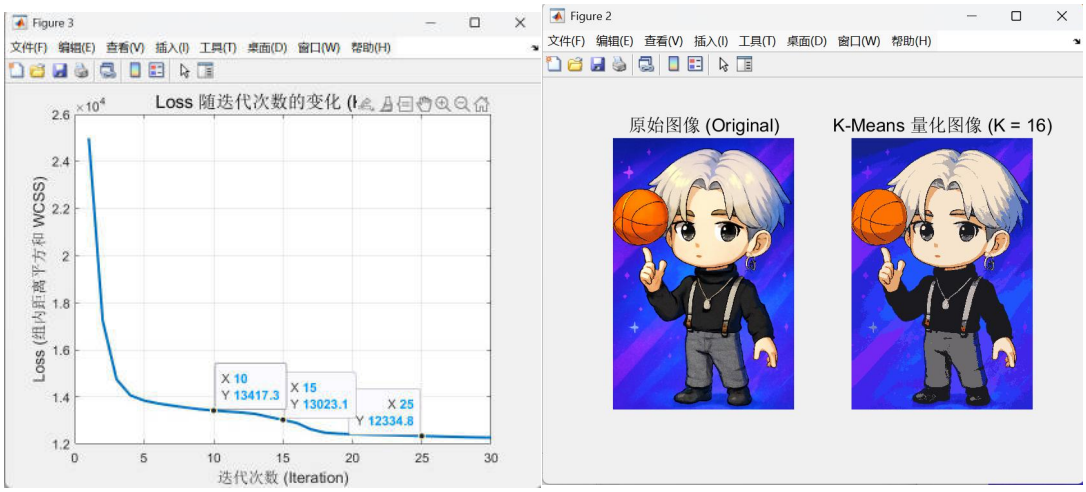
D.max_iter=20



E.max_iter=25



F.max_iter=30



max_iter 太小 (如 5 或 10)：算法可能在 Loss 曲线还未“走平”时就提前终止了。这会导致一个次优的聚类结果，图像的视觉效果可能较差，并且最终的 Loss 值会高于算法完全收敛时的值

max_iter 足够大 (如 20 或 25)：你会看到 Loss 曲线在迭代一定次数后（例如 20 次迭代后）就收敛了（即曲线变平，Loss 值不再下降）。

结论： max_iter 只需要设置得足够大（例如 100），以确保算法有足够的时间收敛即可。一旦收敛，再增加迭代次数（如从 100 增加到 200）对最终的 Loss 值和图像视觉效果没有影响

五、选做题

1．方法核心：

（1）多次运行 K-Means 算法，每次使用不同的聚类数量 K

（2）记录下每次运行收敛后的最终 loss 值 (WCSS)，将 loss 值 vs. K 值绘制成图表

（3）找到这个图表中的“肘部”——即 loss 下降幅度骤减变缓的那个点 。这个点被认为是性价比最高的 K 值。

2．代码分析：

（1）K-means 方法：向上述的 Kmeans 方法封装为 mykmeans

（2）肘部法：

①运行多次 K-Means：循环 K=2 到 20，每次都调用 mykmeans 并存储最终的 loss

```
%% --- 1. 运行多次 K-Means ---
K_range = 2:K_max;
final_loss_values = zeros(length(K_range), 1);

fprintf('开始 K-Means 循环 (K=2 到 %d)...\n', K_max);
for i = 1:length(K_range)
    k = K_range(i);
    disp(['正在运行 K = ' num2str(k)]);
    [final_loss_values(i), ~, ~] = mykmeans(P, k, max_iter_kmeans);
end
fprintf('K-Means 循环完成。\n');
```

②绘制原始的 Loss vs. K 曲线

```
%% --- 2. 绘制原始 Loss vs. K 曲线 ---
figure;
plot(K_range, final_loss_values, 'bo-', 'LineWidth', 2);
title('K-Means 肘部法曲线 (原始数据)', 'FontSize', 14);
xlabel('聚类数量 (K)', 'FontSize', 12);
ylabel('最终 Loss (WCSS)', 'FontSize', 12);
grid on;
```

③实现 Kneedle 算法：

A.归一化：这是非常关键的一步。K 值的范围 (2-20) 和 Loss 值的范围 (例如 10^4 - 10^5) 尺度差异巨大。我们必须先将两者都缩放到 [0, 1] 范围内，才能公平地计算距离

```
% 归一化
x_norm = (K_range - min(K_range)) / (max(K_range) - min(K_range));
y_norm = (final_loss_values - min(final_loss_values)) / (max(final_loss_values) - min(final_loss_values));
```

B.计算距离：我们计算每个归一化后的点$(x_{\text{norm}}, y_{\text{norm}})$ 到归一化的"收尾连线"（从 (0, 1) 到 (1, 0)）的垂直距离

```
distances = abs(x_norm' + y_norm - 1) / sqrt(2);

% 找到距离最大的点
[max_dist, max_idx] = max(distances);
optimal_K = K_range(max_idx);
```

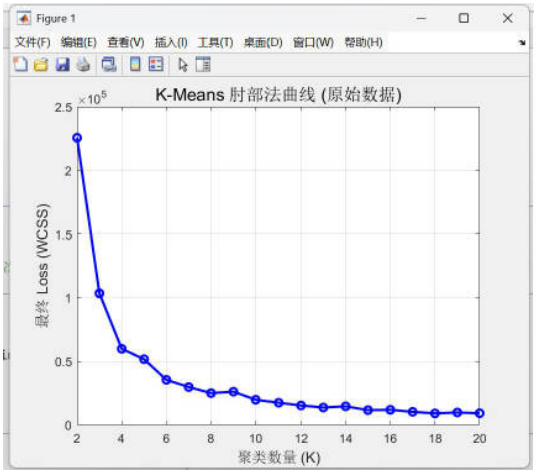
C.找最大值：距离最大的那个点，就是曲线弯曲得最厉害的点，即“肘部”

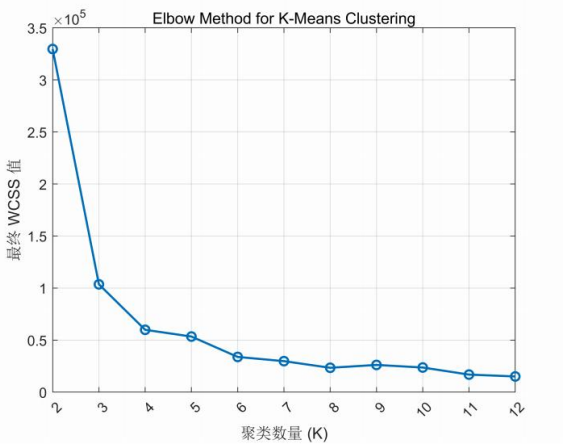
```
figure;
plot(x_norm, y_norm, 'bo-', 'LineWidth', 2, 'DisplayName', '归一化的 Loss');
hold on;
plot([0 1], [1 0], 'r--', 'LineWidth', 1.5, 'DisplayName', '收尾连线');
% 绘制肘部点
plot(x_norm(max_idx), y_norm(max_idx), 'kp', 'MarkerSize', 18, 'LineWidth', 3, 'DisplayName', '检测到的肘部');
% 绘制肘部点到直线的距离
plot([x_norm(max_idx), (x_norm(max_idx) + y_norm(max_idx))/2], [y_norm(max_idx), (x_norm(max_idx) + y_norm(max_idx))/2], 'g-', 'DisplayName',
hold off;

title('Kneedle 肘部法 (归一化数据)', 'FontSize', 14);
xlabel('归一化的 K 值', 'FontSize', 12);
ylabel('归一化的 Loss (WCSS)', 'FontSize', 12);
legend('show', 'Location', 'best');
grid on;
```

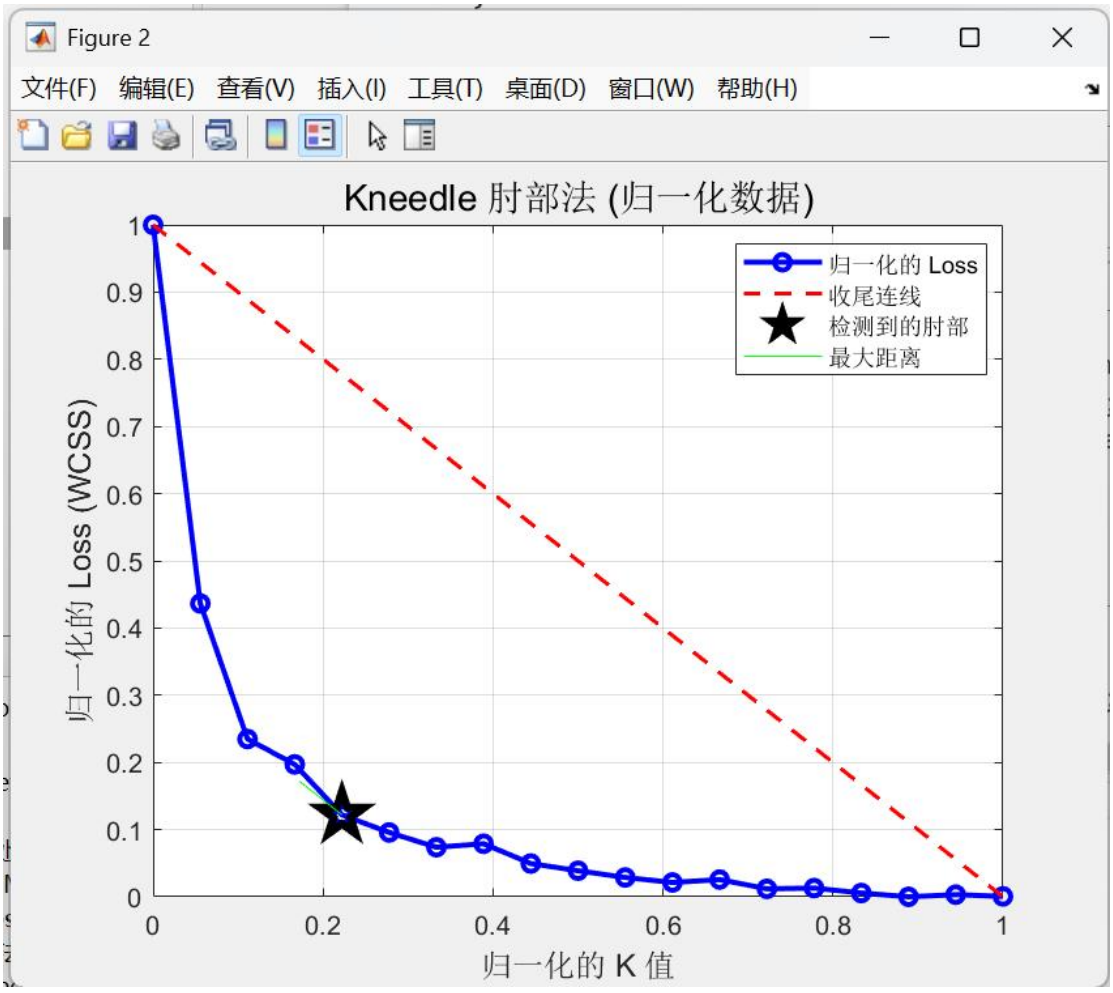
3．结果分析：

（1）原始数据：对比 pdf 中的结果，由于初始化选取的簇是随机的，所以一定程度上会影响最后的结果





（2）归一化数据：归一化后，肘部法求解距离最近，找到离首尾连线最近的那个点，从图中可以看出后，反归一化计算得到 K=6



六、思考与感悟

本次实验让我对 K-Means 这一经典的聚类算法有了远超书本的深刻理解。它不仅仅是一个数学公式或几行代码，更是连接抽象数据与直观视觉的桥梁。实验的核心任务——图像颜色量化，巧妙地将一个视觉艺术问题（减少图像颜色）转换为了一个数据科学问题（在三维 RGB 空间中寻找数据簇）。

在实现 K-Means 算法的过程中，我最大的感悟是其“分块坐标下降法”思想的简洁与高效。这个听起来复杂的优化理论，被完美地拆解为两个直观且交替迭代的步骤：

1．分配簇：固定聚类中心，将每个像素点分配给离它最近的中心。这本质上是在最小化每个点与其“代表色”的距离。

2．更新中心：固定分配方式，将每个聚类中心移动到其所有“子民”（像素点）的平均位置。这又是在为下一轮分配寻找更优的“代表色”位置。 在实验中，当我绘制出 Loss（组内距离平方和）随迭代次数的变化曲线时，清晰地看到了一个单调下降直至收敛的过程。这直观地证明了算法的每一步都是在“改进”结果，最终必然会收敛到一个（至少是局部的）最优解。

本次实验的另一个核心，是对超参数 K 值的探索。K 值的选择，完美地诠释了“模型复杂度”与“性能”之间的权衡。当 K 值很小（如 K=2 或 K=4）时，图像被高度压缩，颜色单一，细节丢失严重，呈现出一种“海报化”的艺术风格；当 K 值增大时，图像细节逐渐丰富，更接近原图，但量化的意义也在减弱。

“肘部法”提供了一种从数据本身出发、寻找“最优 K”的量化思路。绘制 K-Loss 曲线，我发现 Loss 总是随 K 增大而减小，但其下降的“边际效益”在递减。那个下降速率骤然变缓的“肘部”，就是“性价比”最高的 K 值——它是在“保留最多图像信息”和“使用最少颜色”之间找到的最佳平衡点。这让我感悟到，许多工程问题和数据科学问题，寻找的并非理论上的“绝对最优”，而是约束条件下的“最适解”。

最后，实验也让我意识到了 K-Means 的局限性，即它对初始聚类中心的选择非常敏感。不同的随机起点，可能会导致算法收敛到不同的局部最优解，从而产生视觉效果截然不同的量化图像。这启发我，在实际应用中，可能需要多次随机初始化或使用 K-Means++ 等改进策略来寻求更稳定的全局最优结果。

总而言之，这次实验是一次宝贵的实践。它将我从抽象的数学推导，带到了直观的视觉结果面前，让我深刻体会到了算法如何从混乱的数据中“创造”出结构和秩序。